

트랜스포머(Transformer) & GPT 모델

Transformer

- 2017년 코넬 대학 아시시 바스와니(Ashish Vaswani)등의 연구 그룹이 발표한 'Attention is All You Need' 논문에서 소개된 신경망 아키텍처
- 병렬로 입력 시퀀스 처리 → 긴 시퀀스의 경우 RNN보다 훨씬 더 빠르고 효율적 처리
 - 기존의 Sequential Processing이나 Recurrent Connections에 의존하지 않고, token 간 관계를 직접 처리하고 이해할 수 있는 **Self-Attention** 기반
 - 재귀나 합성곱 연산 없이 입력 토큰 간의 관계 직접 모델링 가능
- 대용량 데이터셋에서 효율적 → 데이터 양 많은 기계 번역과 같은 작업에 적합
- 언어 모델링 및 텍스트 분류 작업에 효율적 → 광범위한 자연어 처리 작업에서 높은 효율
- 장기적인 종속성 포함 작업 (ex) 기계번역, 언어모델링, 텍스트 요약

트랜스포머 구조



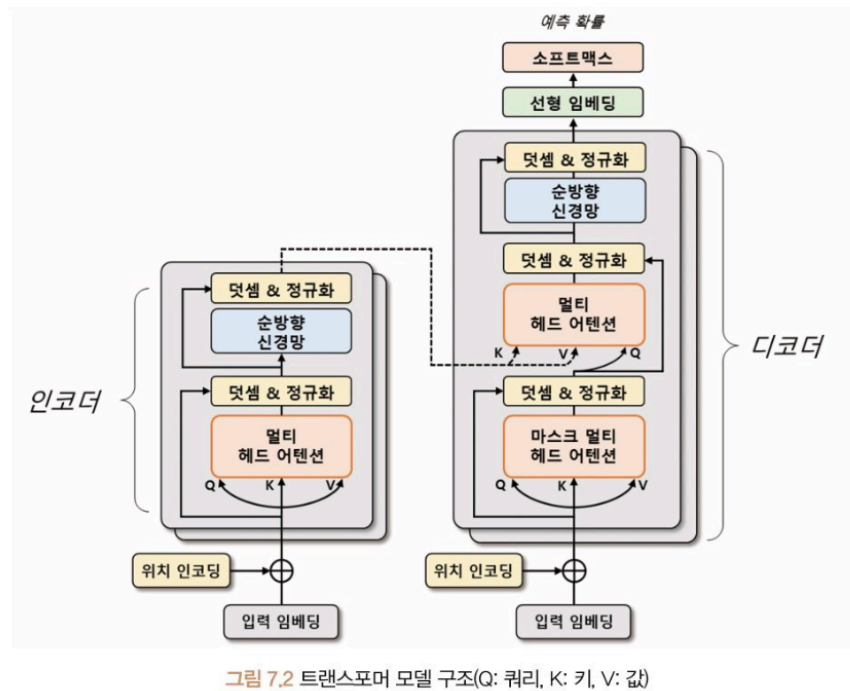
- **Auto-Encoding(인코더)**
 - 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고, 해당 빈칸에 어떤 단어가 적절할지 예측 작업
 - 예측되는 토큰의 양옆에 있는 토큰들을 참조하기 때문에 양방향 과정 = 인코더
- **Auto-Regression(디코더)**
 - 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞추는 작업 수행
 - 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하기 때문에 단방향 과정 = 디코더

표 7.1 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

- **Attention Mechanism** : 인코더와 디코더 간의 상호작용
 - 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥 이해 적절한 출력 생성
 - 인코더 : 입력 시퀀스를 임베딩 고차원 벡터로 변화
 - 디코더 : 인코더의 출력을 입력으로 받아 출력 시퀀스 생성
 - 이때 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중
 - 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지 파악 → 번역, 요약문 생성과 관련된 작업 수행

Transformer 모델 구조(Q / K / V)



- 인코더와 디코더 두 부분으로 구성
- 각각 N개의 Transformer Block으로 구성
 - Transformer Block은 Multi-Head Attention과 순방향 신경망으로 구성됨
- Multi-Head-Attention
 - 입력 시퀀스에서 Q(Query), K(Key), 값(Value) 벡터를 정의해 입력 시퀀스들의 관계를 Self-Attention 하는 방법

input embedding & Positional encoding

- 입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환된 → 입력 시퀀스를 병렬 구조로 처리 → 단어의 순서 정보 제공 X → 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보 모델에 반영 필요

Transformer: Encoder

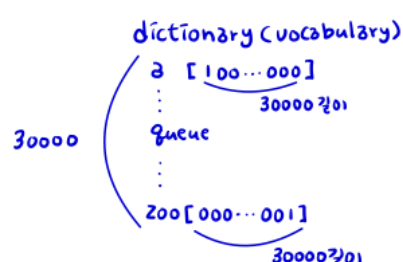
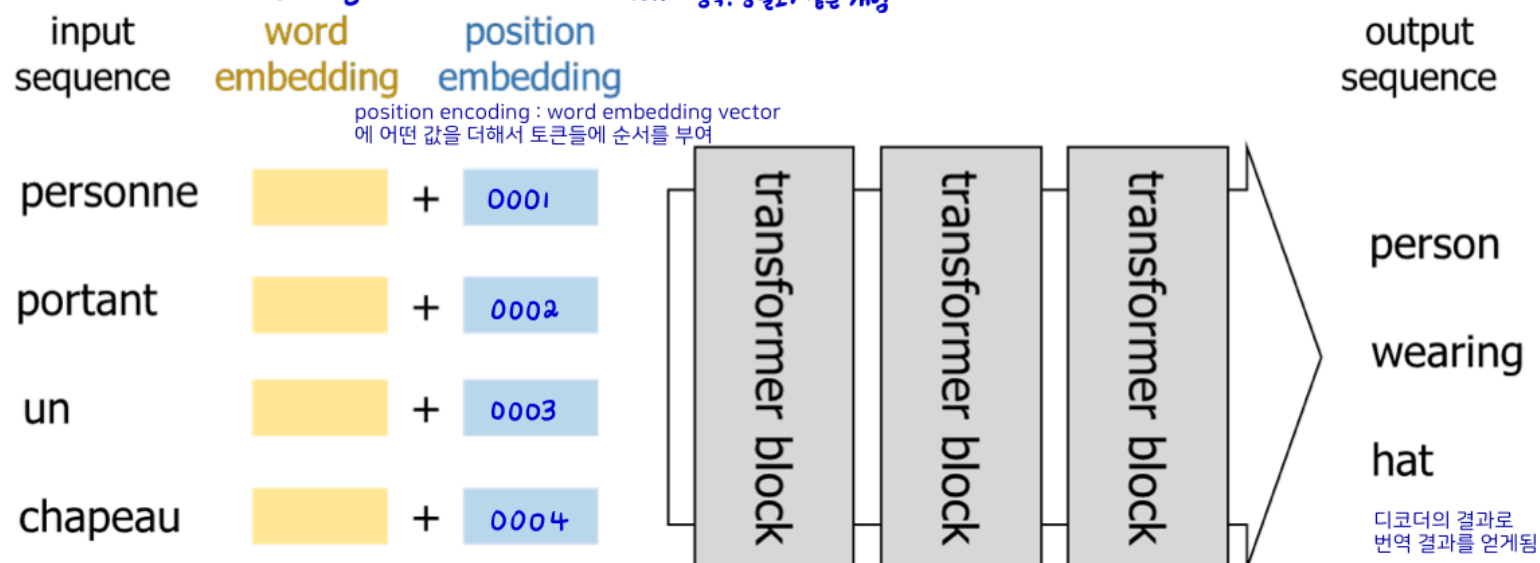
Positional encoding

입력된 단어 벡터들에 위치 정보 부여

- So far, the order of the sequence does not affect result of computations
- Make input vectors have positions

word embedding은 의미 정보를 벡터에 담는 과정

ex) king - man + women = Queen 왕족, 성별과, 같은 개념



dictionary
영어의 모든 단어 포함한다고 가정
이 단어들은 one-hot vector로 표현됨
그래서 우리는 30000차원의 벡터를 가지게 됨
one-hot vector는 단어에 대한 어떤 의미 정보도 담고 있지 않으며 위치정보만 담음
우리는 이 벡터를 사용해 embedding matrix를 생성함
우리는 이 one-hot vector를 word embedding vector로 변환함

이후에 transformer block에 넣고 인코더 디코더를 거쳐

- **Positional Encoding** : 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
 - 각 토큰의 위치를 각도로 표현해 sin 함수와 cos 함수를 사용해 생성
 - 토큰의 위치마다 다른 임베딩 벡터를 사용하기 때문에 각 토큰의 위치 정보 모델이 학습 가능

```
import math
import torch
from torch import nn
from matplotlib import pyplot as plt

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

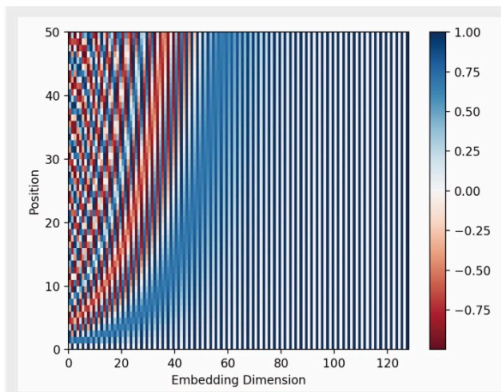
        pe = torch.zeros(max_len, 1, d_model)
        pe[:, 0, 0::2] = torch.sin(position * div_term)
        pe[:, 0, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(0)]
        return self.dropout(x)

encoding = PositionalEncoding(d_model=128, max_len=50)

plt.pcolormesh(encoding.pe.numpy().squeeze(), cmap="RdBu")
plt.xlabel("Embedding Dimention")
plt.xlim((0, 128))
plt.ylabel("Position")
plt.colorbar()
plt.show()
```

출력 결과



- **d_model** : 임베딩 차원 (예: 128차원)
- **max_len** : 처리 가능한 최대 시퀀스 길이
- **dropout** : 과적합 방지를 위한 드롭아웃 확률

```
position = torch.arange(max_len).unsqueeze(1)
```

- **position** : `[0, 1, 2, ..., max_len-1]` 형태의 열 벡터로, 각 토큰의 **위치 인덱스**입니다.

- `.unsqueeze(1)` → `[max_len, 1]` 로 reshap

```
div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
```

- `div_term` : 주기 조절을 위한 계수
- `10000^(2i/d_model)` 의 역수 형태
- `d_model=128` 이라면 `0, 2, 4, ..., 126` 에 대한 값

```
pe = torch.zeros(max_len, 1, d_model)
pe[:, 0, 0::2] = torch.sin(position * div_term)
pe[:, 0, 1::2] = torch.cos(position * div_term)
```

- `pe` : `[max_len, 1, d_model]` 크기의 위치 인코딩 텐서
- 짝수 인덱스 (0, 2, 4, ...) → 사인
- 홀수 인덱스 (1, 3, 5, ...) → 코사인
- 이 구조는 단순 반복이 아닌, 다양한 주기를 가지는 사인/코사인 함수를 조합하여 포지션을 표현

```
self.register_buffer("pe", pe)
```

- 이 위치 인코딩 `pe`는 학습되지 않지만 모델에 포함되어야 할 값이기 때문에 `register_buffer` 로 등록
- 학습 파라미터는 아니지만, `.to(device)` 나 `.state_dict()` 에는 포함됨

위치 인코딩 수식

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos / 10000^{2i/d_{model}})$$

- `pos` : 입력 시퀀스에서 해당 단어의 위치
- `i` : 임베딩 벡터의 차원 인덱스
- 차원의 인덱스가 짝수라면 첫 번째 `sin` 함수 수식을 적용하며, 홀수라면 두 번째 `cos` 함수 수식을 적용
- $\frac{1}{10000^{2i/d_{model}}}$: 각도 정보를 변환하기 위한 스케일링 인자로 각 위치에 대한 주기적인 신호를 생성
- `pe` 변수의 텐서 차원은 `[50,1,128]` → `[최대시퀀스, 1, 입력임베딩의 차원]`
- 산출된 위치 인코딩은 입력 임베딩과 더해진 후 드롭아웃이 적용

특수 토큰

- 입력 시퀀스의 시작과 끝을 나타내거나 Masking 영역으로 사용됨
- BOS(Beginning of Sentence), EOS(End of Sentence)
- UNK(Unknown): 어휘 사전에 없는 단어(모르는 단어) → 모델이 이전에 본 적 없는 단어 처리할 때
- PAD(Padding) : 모든 문장을 일정한 길이로 맞추기 위해 사용 → 짧은 문장의 빈 공간을 채울 때 사용

- 특수 토큰을 활용한 문장 토큰 배열

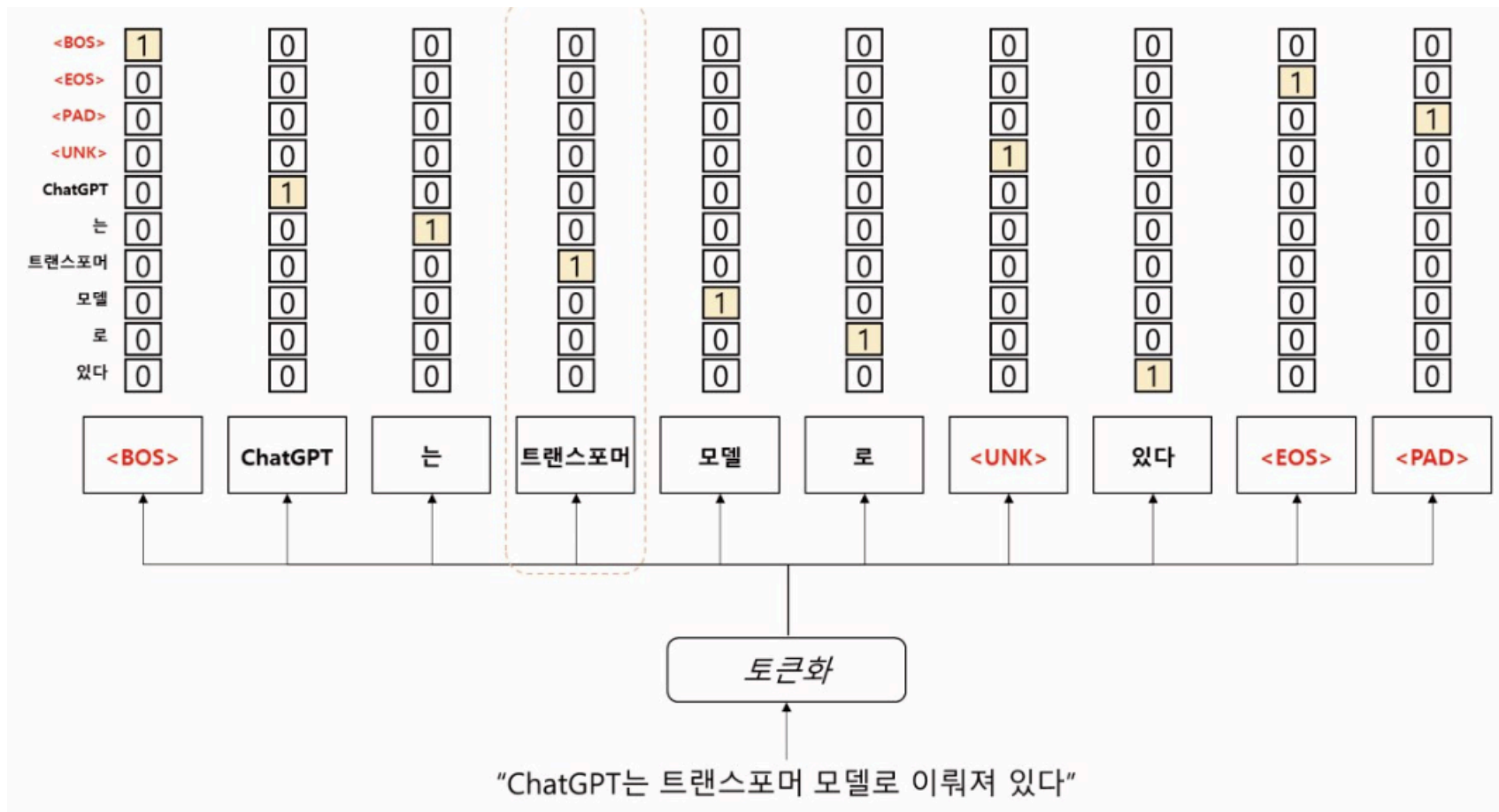


그림 7.3 특수 토큰을 활용한 문장 토큰 배열

⇒ 이렇게 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 One-Hot Encoding

• 입력 임베딩 산출 과정

- (ex) '트랜스포머'라는 One-Hot vector가 input Embedding으로 변환되는 과정

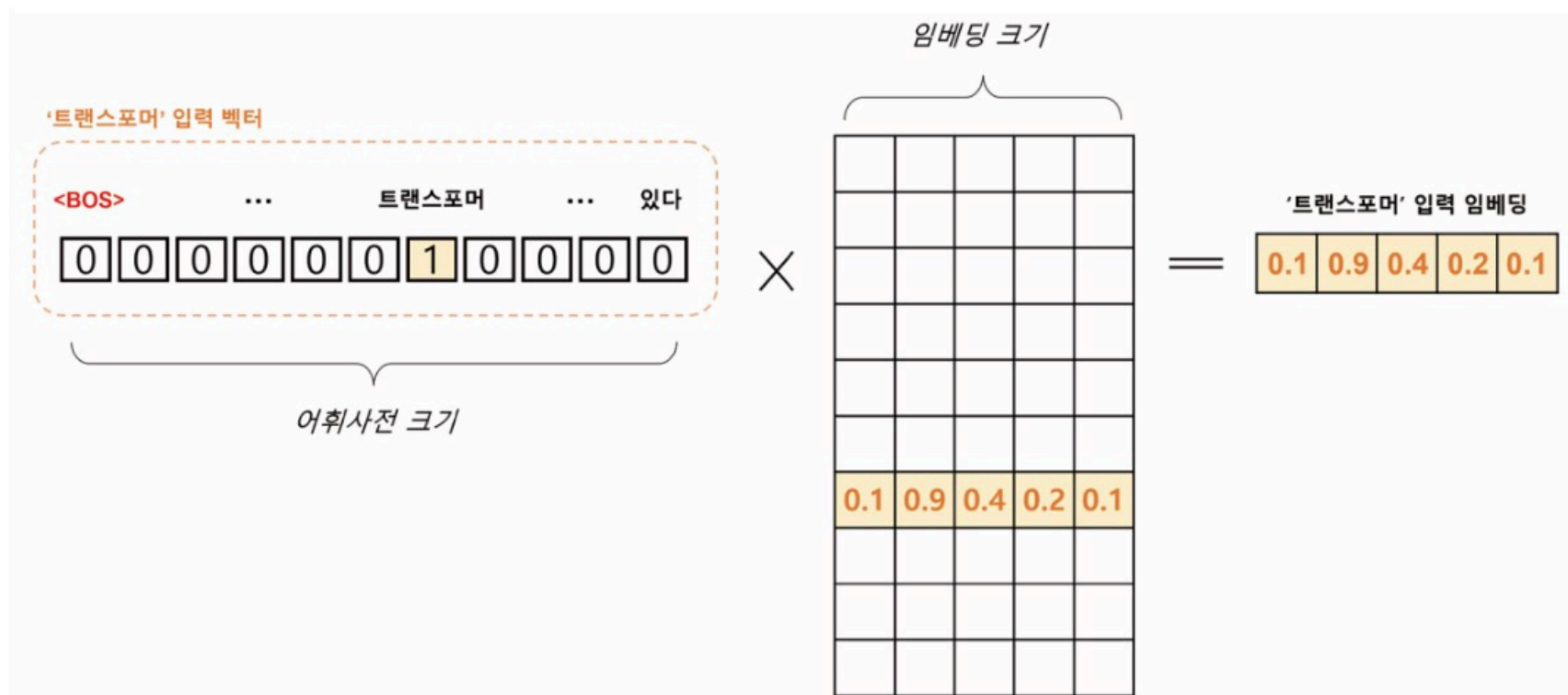


그림 7.4 입력 임베딩 산출 과정

- One-Hot Vector의 크기 : $[1, V]$
 - V : 어휘 사전 크기
 - d : 입력 임베딩 차원
- 이 One-Hot Vector는 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환됨
 - ⇒ N 개의 문장이 최대 S 개의 토큰 길이를 가질 때 $[N, S, V]$ 크기의 One-Hot Vector Tensor는 $[N, S, d]$ 크기의 임베딩 텐서로 변환됨
 - 입력으로 사용되며, 트랜스포머의 모든 계층에서 공유되는 텐서로 사용됨

Transformer Encoder

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행

Transformer

Model architecture (1)

- Get rid of the sequential parts
 - Highly scalable, highly parallelizable
- Encoder-decoder structure with only attention and fully-connected layers
 - No recurrence or convolutions
- Stack encoder-decoder N_{layers} times
- Hyperparameters:
 - d_{model} : embedding vector size
 - N_{layers} : number of layers
 - N_{heads} : number of heads
 - d_{ff} : number of hidden units in feed forward network

이 문장을 몇 개의 조각으로 나누는 방법
그리고 이걸 어떻게 벡터로 바꾸는 지도 매우 중요한 주제
"고양이가": 하나의 token = transformer의 하나의 input, "밥을": 두번째 token, "먹는다": 세번째 token
→ 즉, 3개의 token을 tokenizing해야함

이 토큰들은 자연어이기 때문에 의미(semantic)나 문맥 정보(context information) 담고 있지 않으며
그냥 고유한 unicode text

이 token들을 vector로 변환해야함 (attention은 vector를 input으로 받기 때문)
vector로 변환하는 과정 : input embedding, word embedding, 일반적으로는 token embedding이라고 함

input embedding block은 자연어 token을 vector로 바꾸는 작업을 수행하고
vector는 해당 언어에 대한 약간의 context information을 포함하게 됨

(3)~(6) 까지가 Transformer의 한 층
CNN에서 Conv Layer 여러개 쌓는 것 처럼
Transformer에서도 Layer를 여러개 쌓게 됨

Encoder Block 안에는 Multi-Head Attention과 Feed-Forward Network가 존재하고
출력은 다시 다음 Layer로 전달됨

여기서는 Encoder Block을 3번 쌓은 구조를 보여주고 있음

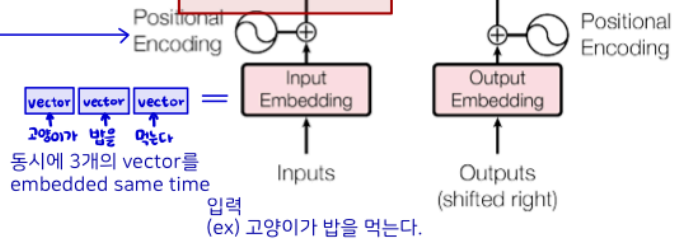
원래 Transformer 모델은 ML을 위해 고안되었음
→ 따라서 encoder decoder가 존재

다시 Add연산(residual(skip) connection for gradient vanishing 약화)
layer Normalization도 다시 수행

우리는 입력으로 부터 토큰 간의 관계를 담은 벡터를 얻었고, 이 벡터는 FF layer에 입력됨
FF Layer는 일방적인 MLP enhance representation

Add 연산(skip connection)을 하고 layer Normalization 수행(CNN의 BN과 유사하지만 동작 방식은 다름)

- ③ self-attention layer
동시에 multi-head attention에 들어감/ 토큰 간의 관계를 포착/representation
- ② 3가지 token 동시에 positional Encoding을 적용
self-attention은 단어의 순서를 고려하지 않기 때문에 우리는 위치 정보를 추가적으로 주입함



①: 우리는 3개의 token을 동시에 Transformer에 입력함

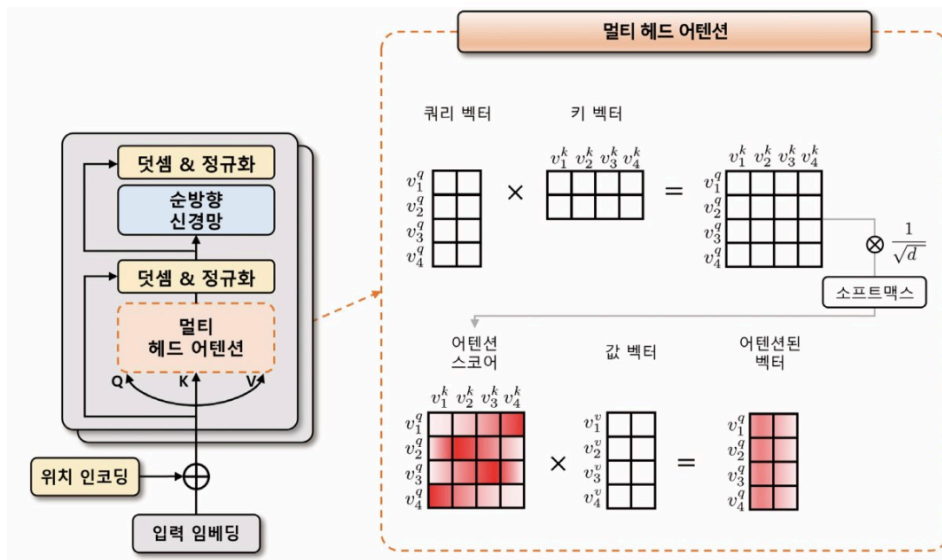
Encoder 과정

1. 여러개의 token을 동시에 Transformer에 입력 → 여러 vectors를 embedded at the same time
2. 여러 vectors 동시에 Positional Encoding을 적용
 - self-attention은 단어의 순서를 고려하지 않기 때문에 → 우리는 위치 정보를 추가적으로 주입
3. Multi-Head Attention : self-attention layer
 - 동시에 multi-head attention에 들어감
 - 토큰 간의 관계를 포착 → representation
4. Add 연산(skip connection)하고 Layer Normalization 수행
5. 얻은 vectors는 Feed Forward Layer에 입력됨
 - enhance representation
6. 다시 Add 연산 for gradient vanishing 약화 + Layer Normalization도 다시 수행

⇒ 3 ~ 6 : Transformer의 한 층 → Layer를 여러개 쌓음

⇒ Encoder Block 안에는 Multi-Head-Attention과 Feed-Forward Network가 존재하고 출력은 다시 다음 Layer로 보내짐

트랜스포머 인코더 블록 연산 과정



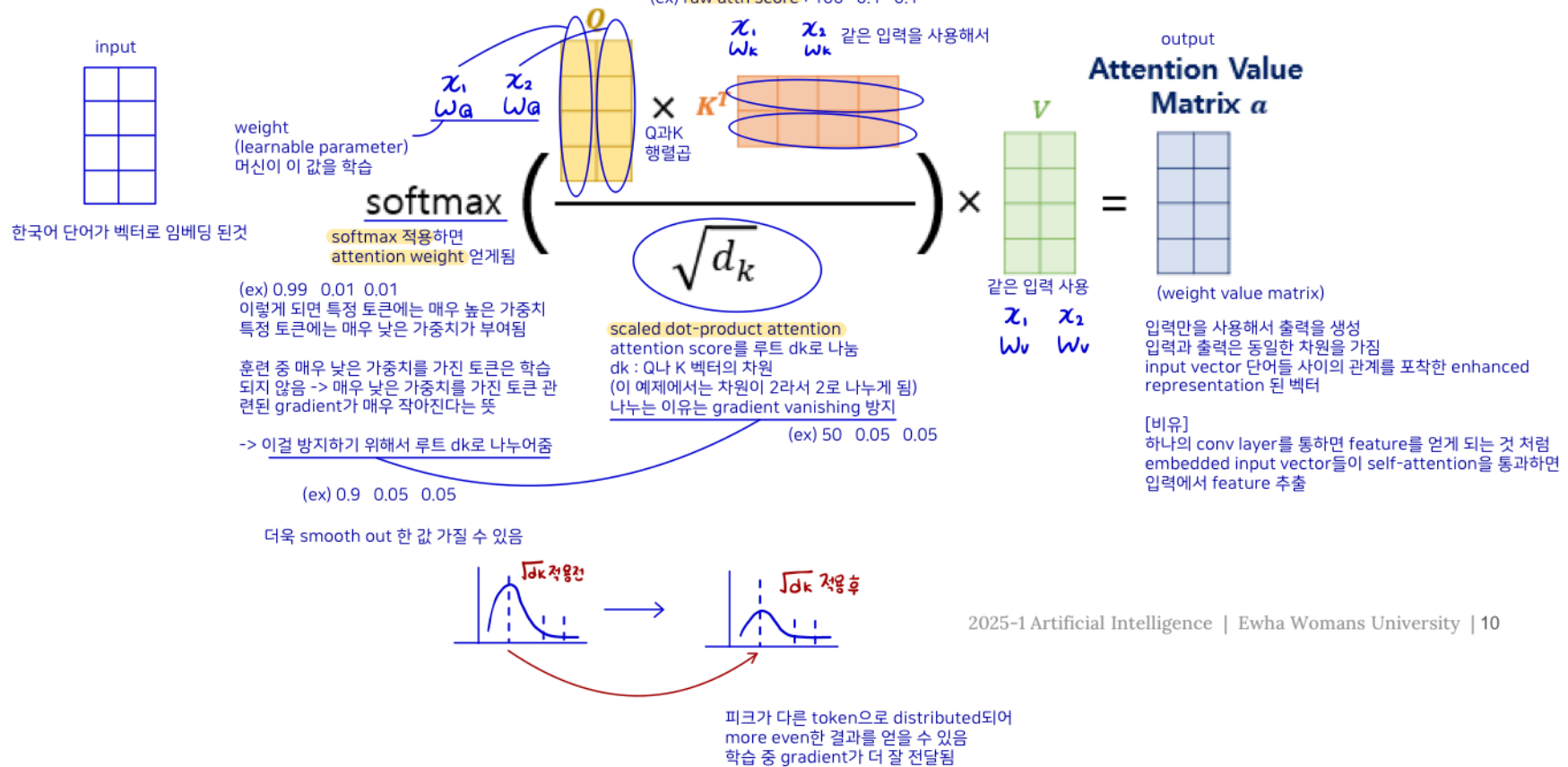
Transformer: Encoder

Scaled dot-product attention

- Large similarities will cause softmax to saturate and give vanishing gradients
- d_k : vector size of Q and K

예시에서는 2개의 input vector를 가짐

attention mechanism의 수식
: 두개의 벡터의 곱 = Query x Key
(ex) raw attn score: 100 0.1 0.1



2025-1 Artificial Intelligence | Ewha Womans University | 10

Transformer Decoder

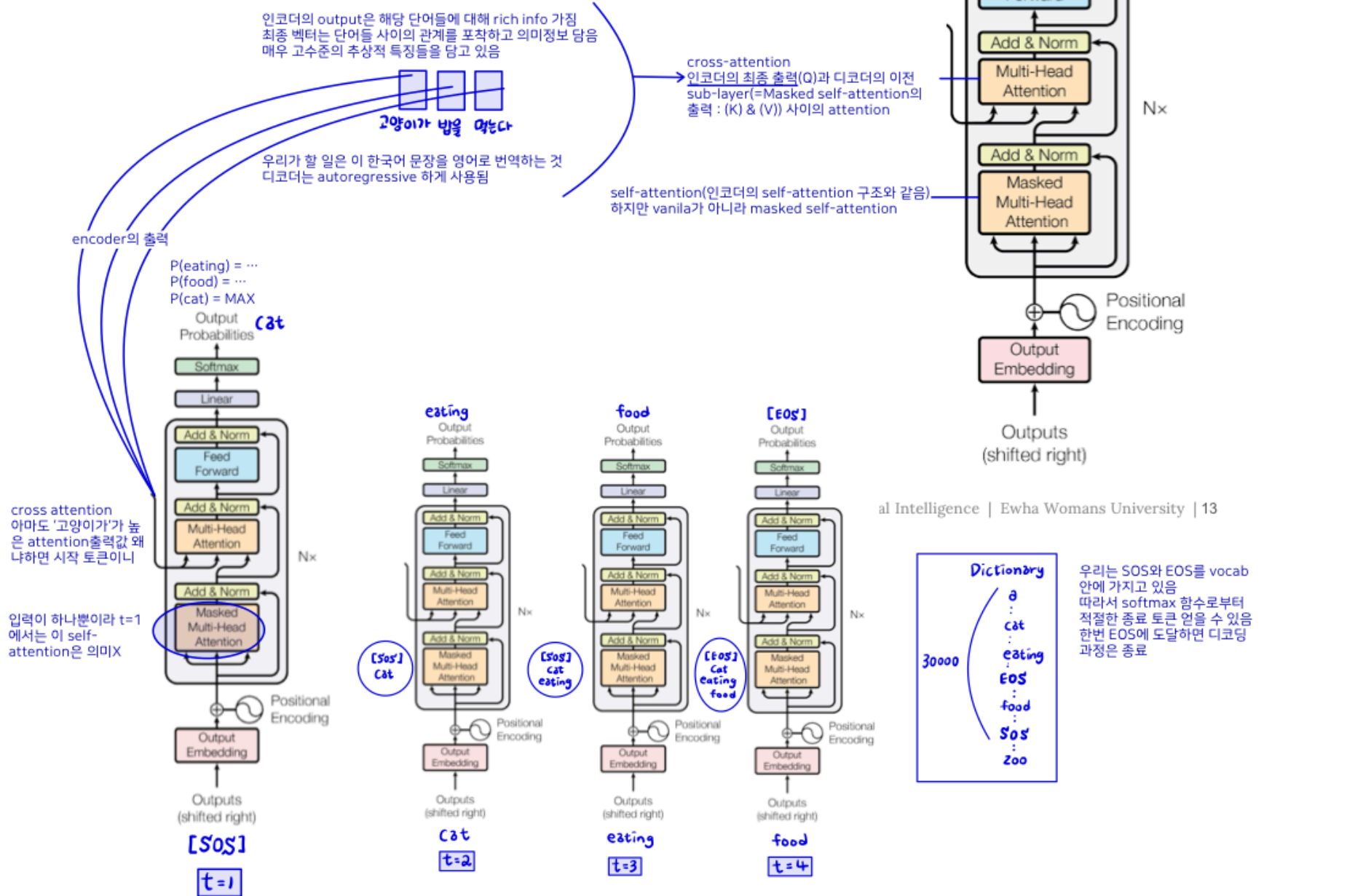
Transformer: Decoder

google brain에 의해서 propose 되었음

Architecture

decoder는 autoregressive

- Decoder has the following key components:
 - Masked self-attention
 - Cross attention



1. Transformer Decoder의 첫번째 Attention Layer

- Decoder에는 Masked-Multi head Attention이 있음 (이것도 역시 self-attention임)
- Autoregression으로, 출력을 한 번에 다 생성하지 않고 순서대로 생성됨 왼쪽에서 오른쪽으로 하나씩 생성
 - 입력을 한꺼번에 받을 수 있지만 출력은 시간에 따라 순차적으로 생성
 - 따라서 Decoder는 아직 생성되지 않은 미래 토큰을 보면 안됨(보면 cheating)
 - Masked Attention : 아직 생성하지 않은 단어를 보지 않고 기존 단어만으로 다음 단어를 예측
 - Self attention과 거의 같지만, Softmax를 적용할 때 mask를 적용해서 미래 단어의 점수를 minus 무한대로 만들
 - Softmax 결과가 0이 되기 때문에 그 단어는 전혀 영향을 미치지 못함

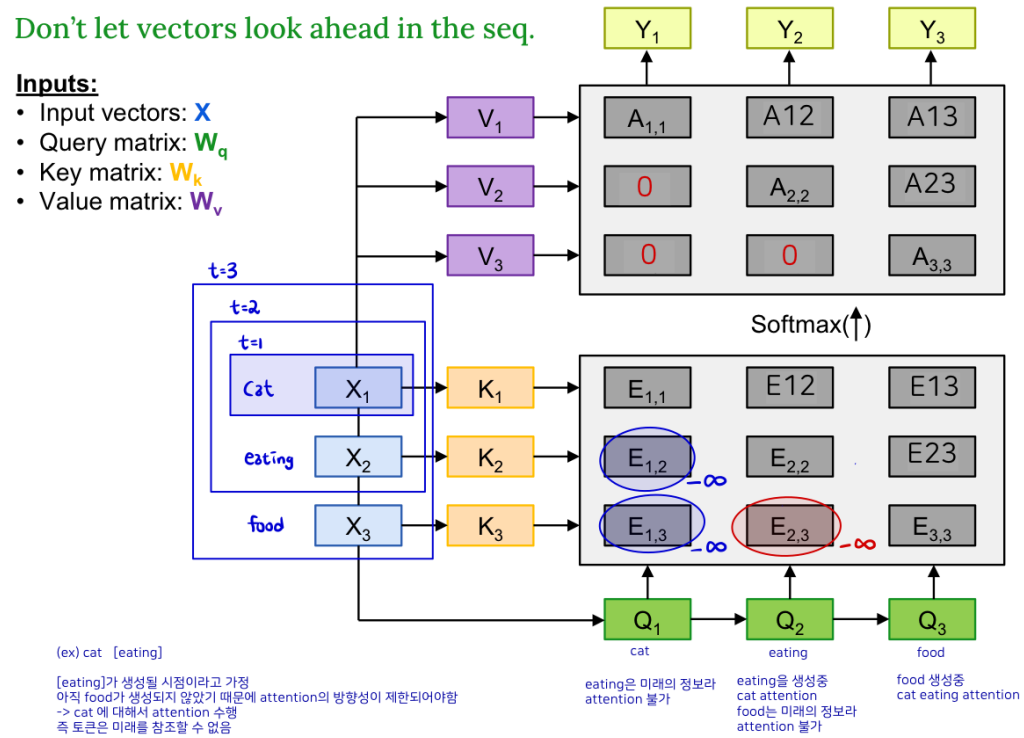
2. 두번째 Attention Layer : Cross- Attention

- Encoder의 출력과 Decoder의 출력 사이에서 Attention이 발생 : Cross Attention
 - Encoder의 출력 : Key, Value
 - Decoder의 출력 : Query
- (ex) 입력 문장은 "personne portant un chapeau"라고 하고 Decoder는 "person wearing"을 생성하려고 한다면
 - Decoder는 "person", "wearing"을 가지고 있고, 이것을 Query로 사용해서 Encoder의 모든 출력(Token)들을 Key와 Value로 삼아 그 중 어떤 입력 단어가 가장 관련이 있는지를 판단하게 됨 : **Cross-Attention**

3. 마지막으로 Feed-Forward Layer가 들어가고, Residual connection과 Layer Normalization도 똑같이 적용됨

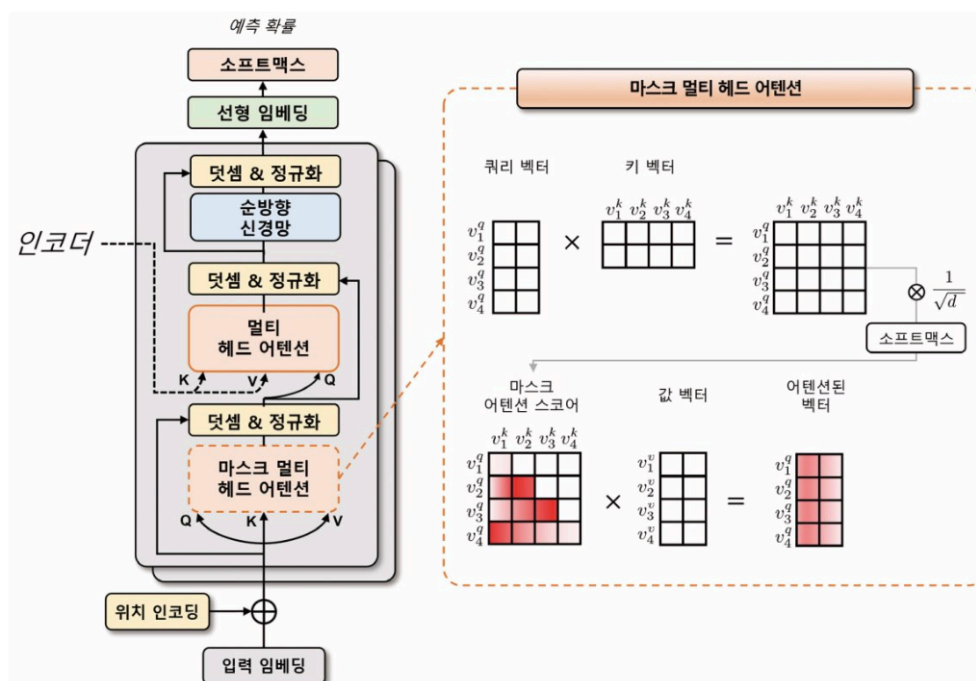
- **Masked Self-Attention Layer**

■ Masked Self-Attention Layer 미래를 못 보도록 masking



- (ex) cat [eating] → [eating]이 생성될 시점이라고 가정
 - 아직 Eating이 생성되지 않았기 때문에 attention의 방향성이 제한되어야함
 - cat에 대해서 attention 수행 ⇒ 토큰은 미래를 참조할 수 없음
 - 이러한 마스크를 적용하면 셀프 어텐션에서 현재 위치 이전의 단어들만 참조 가능 → 인과성이 보장
 - 마스크 영역에 굉장히 작은 값인 -inf 마스크를 더해줌 → 해당 영역 어텐션 스코어값이 0에 가깝
 - 소프트맥스 계산 시 해당 영역 어텐션 가중치가 0 → 마스크 영역 이전 입력 토큰들에 대한 정보 참조X
- ⇒ 마스크 멀티 헤드 어텐션 모듈은 인과성을 보장하면서 셀프 어텐션을 수행

트랜스포머 디코더 블록 연산 과정



- 디코더의 멀티 헤드 어텐션 구조
 - 디코더의 **멀티 헤드 어텐션**에서는 **타겟 데이터**가 쿼리(Query) 벡터로 사용
 - 인코더의 출력(소스 데이터)가 키(Key) 벡터 및 값(Value) 벡터로 사용
 - 쿼리 벡터는 타겟 데이터의 **위치 정보**를 포함한 임베딩 + 위치 인코딩으로 구성됨

- 어텐션 계산 과정

1. 쿼리, 키, 값 벡터를 이용해 어텐션 스코어 맵을 계산
2. 소프트맥스 함수를 통해 어텐션 가중치를 구함
3. 가중치와 값 벡터를 곱해 최종 어텐션 결과를 얻음
4. 이 결과는 디코더의 출력으로 사용됨

- 마스킹의 역할
 - 디코더의 멀티 헤드 어텐션은 인코더의 멀티 헤드 어텐션과 거의 동일하지만, **인과성 보장**을 위해 마스킹(masking)이 적용됨
 - 마스킹은 현재 시점보다 **미래 위치를 참조하지 못하도록** 방지함
- 트랜스포머 디코더 블록 구성
 - 인코더처럼 **여러 개의 디코더 블록**으로 구성됨
 - 각 디코더 블록의 출력은 다음 디코더 블록의 입력으로 전달됨
- 시점 정보 활용
 - 디코더는 이전 시점의 정보를 현재 시점에 활용할 수 있음
 - 디코더 블록 출력 텐서 $[N, S, d]$ 에 대해 **선형 변환 + 소프트맥스**를 적용해 **각 타깃 위치의 예측 확률**을 계산

• 인과성을 고려한 디코더 입력과 출력 예시

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

GPT (Generative Pre-trained Transformer) 모델

- **GPT**는 2018년 OpenAI에서 발표한 **트랜스포머 기반의 언어 모델**로, 가장 널리 알려진 자연어 처리 모델 중 하나
- 이 모델은 ELMo(Embeddings from Language Model)와 같이 **말뭉치로 사전 학습된 모델**이며, 다양한 downstream 작업에서 우수한 성능을 보임
- **GPT의 구조와 특징**
 - GPT는 **트랜스포머의 디코더**를 여러 층 쌓아 만든 언어 모델
 - **트랜스포머 인코더**는 입력 문장의 특성을 추출하는 데 중점을 두는 반면, **GPT의 디코더**는 입력 문장과 출력 문장 간의 관계를 이해하고 다음 단어를 예측하는 데 초점을 둠
 - 이런 구조 덕분에 GPT는 **자연어 생성과 같은 작업**에서 높은 성능
- GPT는 대규모 텍스트 데이터셋을 이용해 **사전 학습(pretraining)** 된 후, **자연어 처리 작업에 맞춰 미세 조정(finetuning)**
- 자연어 생성, 기계 번역, 질의응답, 요약 등 **다양한 NLP 작업**에 활용되며, 특히 적은 양의 데이터로도 높은 성능을 발휘

- GPT-1 (2018) : 최초의 GPT 모델 → 약 **1억 2천만 개의 파라미터**(매개변수)
- GPT-2 (2019) : 두 번째 모델로 성능이 크게 향상됨 → 약 **15억 개의 파라미터**
- GPT-3 (2020) : 약 **1,750억 개의 파라미터**를 가진 초거대 모델
 - 매우 뛰어난 문장 생성 능력을 보임
 - **대규모 연산 자원**이 필요

• GPT-3.5 (InstructGPT)와 RLHF

- OpenAI는 **GPT-3.5** 모델에서 **RLHF (Reinforcement Learning from Human Feedback)** 기법을 적용
- RLHF
 - 모델이 생성한 출력 결과물을 **인간의 피드백**으로 평가하여 보상 신호로 학습
 - 기존 학습 방식보다 더 **정확하고 효율적인 학습**이 가능
 - 약 **60개의 태스크** 기준으로 높은 성능을 기록
- **2022년 10월**, GPT-3.5를 기반으로 한 **ChatGPT 서비스**가 공개 → 자연스러운 대화가 가능하며 지속적으로 개선됨
- **2023년 3월**, **GPT-4**가 발표됨 → 텍스트뿐만 아니라 **이미지 데이터도 함께 학습**하는 모델 → 복잡한 멀티모달 작업도 수행 가능

GPT1

- **GPT-1**은 트랜스포머 구조 기반의 **단방향(Unidirectional)** 언어 모델
- 단방향 언어 모델이란, **현재 위치에서 이전 단어들만을 참조**해 다음 단어를 예측하는 방식의 모델
- 이전 단어들을 **차례로 입력**하면서 다음 단어를 예측하는 방식으로 작동
- 계산량이 줄어들기 때문에 → 학습 속도가 빠르고, 모델의 크기를 작게 유지할 수 있다는 **장점**

구조적 특징

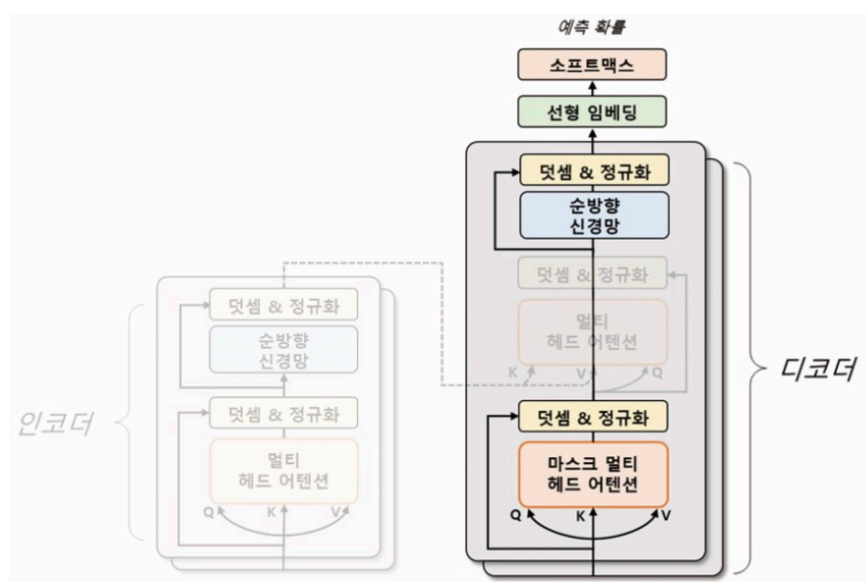


그림 7.7 트랜스포머와 GPT-1 모델 비교

- 트랜스포머 구조 중 **디코더 부분만 사용**하고, 인코더-디코더 간 어텐션 연산은 제외
- 또한 디코더의 두 번째 다중 헤드 어텐션(Multi-head Attention)은 사용하지 않음

⇒ **마스크 멀티 헤드 어텐션**만 사용 (미래 단어 정보 차단)

⇒ **12개의 헤드, 12층 트랜스포머 디코더** → 총 약 **1억 2천만 개의 파라미터**

- 학습 데이터 및 방식
 - 약 **4.5GB 분량의 BookCorpus** 데이터를 사용하여 사전 학습
 - 입력된 문장의 앞부분이 주어지면 **뒷부분 문장을 자동 예측**하도록 학습됨
- 학습의 장점
 - 별도의 레이블(label)이 필요 없는 **비지도 학습** 방식

- 대규모 컴퓨팅 자원만 있으면 비용 부담 없이 학습 가능

GPT-1의 사전 학습 구조

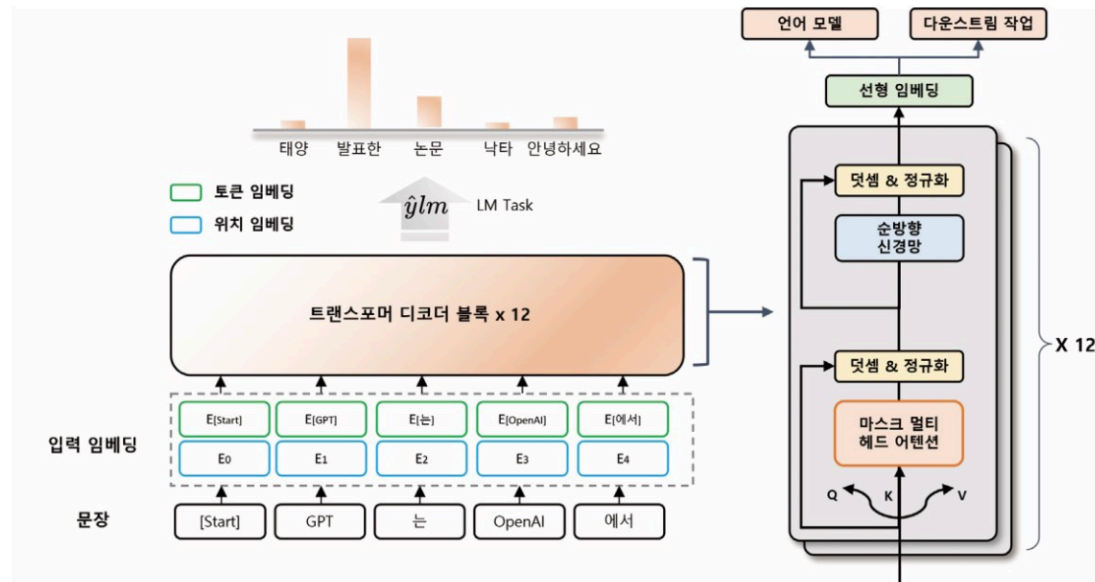


그림 7.8 GPT-1의 사전 학습 작업 구조

- GPT-1은 입력 문장의 **일부분만 보고 다음 토큰을 예측**하도록 사전 학습됨
- 입력 문장을 **일정 길이의 블록**으로 나눈 뒤, 각 블록의 첫 번째 토큰부터 **순차적으로 다음 토큰을 예측**
- 이 방식은 모델이 **장기적인 문맥 이해** 능력을 갖추도록 도움
- 예시: 문장 **[Start] GPT 는 OpenAI 에서** 가 주어졌을 때, → 순차적으로 각 토큰의 다음 단어를 예측하도록 학습됨

- 입력 임베딩 구성
 - **입력 임베딩 = 토큰 임베딩 + 위치 임베딩**
 - **토큰 임베딩**: 각 토큰을 고정된 벡터로 매핑
 - **위치 임베딩**: 각 토큰의 위치 정보를 나타내는 벡터

- 보조 학습(Auxiliary Learning)
 - GPT-1은 미세 조정을 위한 **다운스트림 작업**에서도 **언어 모델을 보조 학습으로 활용**할 수 있음
 - 보조 학습
 - **주어진 주요 작업 외에, 동시에 부가적인 학습 작업을 병렬로 수행하는 것**
 - **주요 작업 성능 향상에 도움이 되는 정보를 학습하게 함**

GPT-1 다운스트림 작업 구조

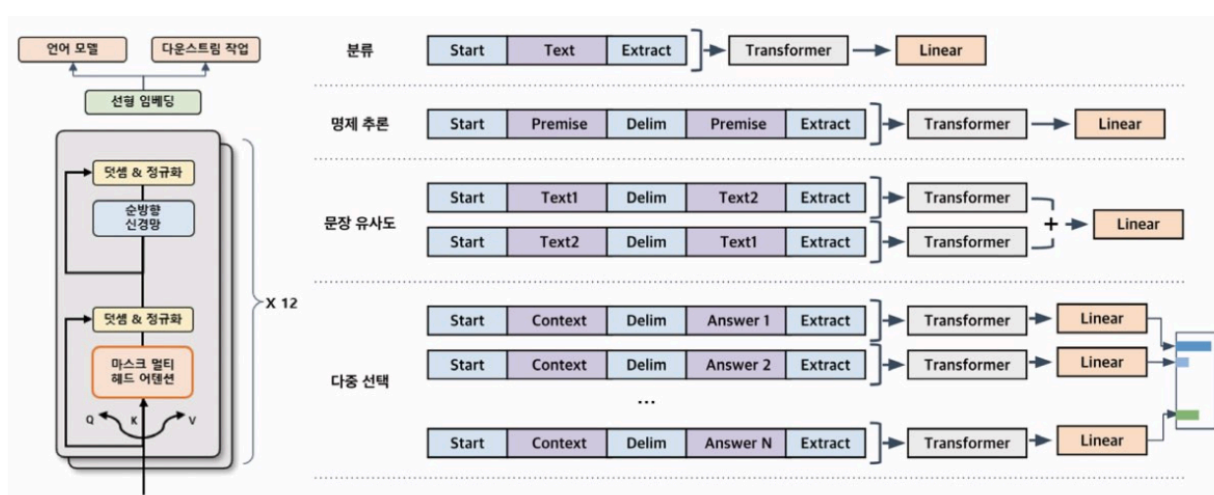


그림 7.9 GPT-1의 다운스트림 작업 구조

입력 구성	작업 설명
Start + Text + Extract	문장 일부로부터 중요한 정보 추출

입력 구성	작업 설명
Start + Context + Delim + Text + Extract	컨텍스트를 기준으로 텍스트에서 핵심 추출
Start + Context + Delim + Premise + Delim + Text + Extract	전제와 조건을 기준으로 텍스트 분석
Start + Context + Delim + Question + Delim + Answer + Extract	질문에 대한 정답 생성 및 추출

- 특수 토큰 사용
 - `<start>` : 문장 시작
 - `<delim>` : 문장 간 구분
 - `<extract>` : 문장 끝/결과 추출
- 손실 함수 및 학습
 - 다운스트림 작업에선 목적에 맞게 모델을 정밀 조정해야 하므로 **일반 손실 함수 외에 추가 손실 함수**가 필요
 - GPT-1은 **두 개의 손실 함수**를 동시에 사용하여:
 - 보조 학습을 통한 언어 모델 개선
 - 다운스트림 작업에 필요한 정보 습득

GPT2

- 구조 확장 및 성능 향상되었음 : GPT-1은 입력 문장을 **최대 512 토큰** 길이까지 처리했지만, **GPT-2는 최대 1,024 토큰 길이**까지 처리 가능해짐
- 디코더 층 수도 증가: [GPT-1] **12개 디코더 블록** → [GPT-2] **48개 디코더 블록** 사용 → 더 깊은 네트워크로 복잡한 패턴 학습 가능
- 매개변수 수 차이 : [GPT-1] 약 1.2억 → [GPT-2] 약 15억 → 약 **10배 이상 증가**함으로써 **표현력 및 생성 성능 향상**

GPT-2의 모델 구조

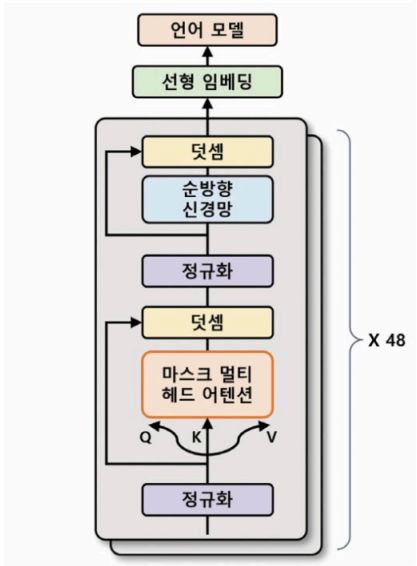


그림 7.10 GPT-2의 모델 구조

- GPT-2도 GPT-1과 마찬가지로 **인코더 없이 디코더만 쌓은 구조**
 - 선형 임베딩 → 순방향 신경망 (Feedforward) → 마스크 멀티 헤드 어텐션 (미래 정보 차단) → **정규화 및 던셉 모듈** 반복
- ⇒ 위 구조가 **48층** 반복됨
- 사전 학습 데이터 : GPT-2는 **미국 웹사이트**에서 크롤링한 약 **8백만 개 문서**, 약 **40GB 텍스트**로 학습됨
 - 더 많은 데이터와 더 깊은 구조 덕분에 **보다 자연스러운 문장 생성 가능**, 더 다양한 분야에서의 활용 가능
- 제로샷 학습 가능 : GPT-2는 별도의 미세 조정 없이도 **다운스트림 작업에 직접 사용 가능**
 - 예시
 - `"GPT는 놀라워요"` → `"GPT is amazing"` 과 같은 **형태의 문장을 학습시킨 후**,

- "번역 한글 문장 =" 을 입력하면 자동으로 영어 번역이 출력되는 형식으로 작동

GPT3

- **GPT-3**는 GPT-2와 **동일한 트랜스포머 디코더 구조**를 유지하지만, 모델 파라미터 수에서 큰 차이를 보임 → GPT-2보다 **약 116배 많은 파라미터**를 보유
 - **96개의 디코더 층, 96개의 멀티헤드 어텐션**
 - 어텐션 계산 비용을 줄이기 위해: **희소 어텐션(Sparse Attention)** + 일반 어텐션을 혼합 사용
 - 토큰 임베딩 벡터 크기: **1,600차원**
 - **최대 입력 길이 2,048개 토큰** 지원 → GPT-2의 1,024개보다 **두 배 길이의 문장 처리 가능**
-
- **약 45TB 분량의 웹 크롤링, 위키백과, 서적 등 대규모 텍스트**로 사전 학습
 - 질문응답, 번역, 요약, 문서 생성 등 **다양한 다운스트림 작업**에서 고성능
 - **사전 미세 조정 없이도** 높은 성능 발휘 가능

GPT-3의 사용 방식: 프롬프트 기반 학습

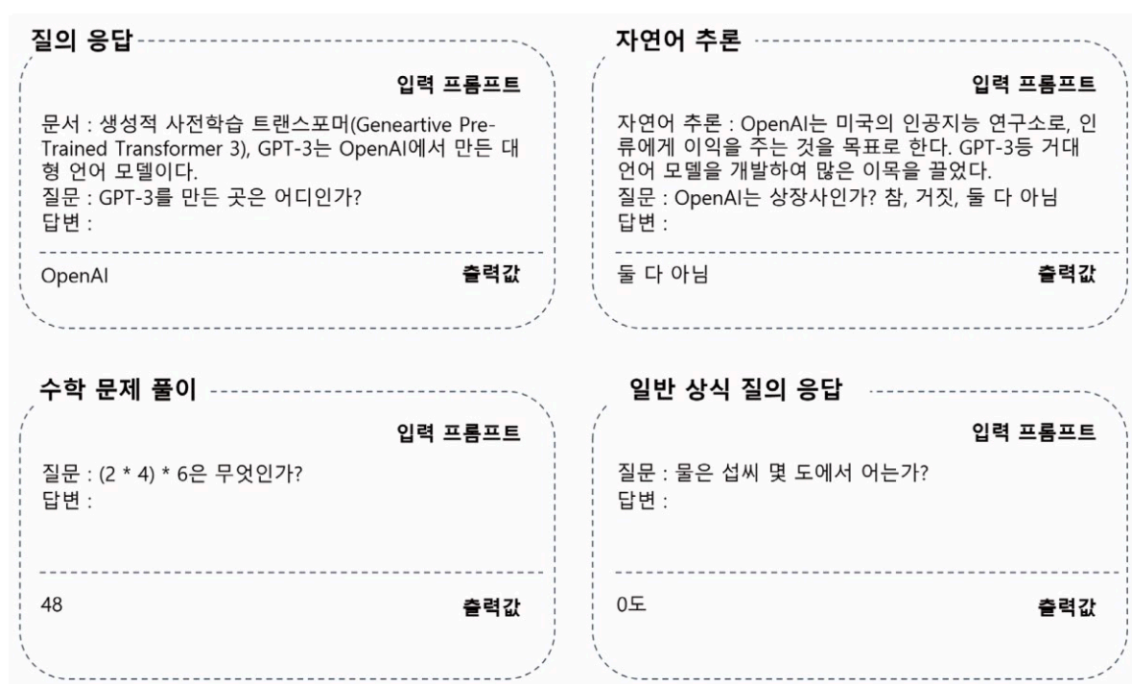


그림 7.11 GPT-3의 다운스트림 작업 프롬프트 예시

작업 유형	입력 프롬프트	출력
질의응답	GPT-3를 만든 곳은 어디인가요?	OpenAI
자연어 추론	OpenAI는 상장사인가? 참, 거짓, 둘 다 아님	둘 다 아님
수학 문제	(2 * 4) + 6은 무엇인가요?	14
상식 질의응답	물은 섭씨 몇 도에서 끓는가?	100도

- 다양한 도메인과 작업 유형을 단일 프롬프트로 수행 가능
-
- 한계와 유의점
 - 모델 크기 때문에 **응답 속도 느리고 비용이 많이 듭**
 - **소규모 기업 및 개인 사용자는 접근 어려움**
 - **사전 학습된 데이터에 기반하여 작동하기 때문에, 데이터가 편향되었거나 부정확할 경우 오류 발생 가능**

GPT 3.5 (InstructGPT)

- **GPT-3.5**는 **GPT-3의 구조는 그대로 유지하면서도**, 새로운 학습 방식인 RLHF (Reinforcement Learning from Human Feedback)를 도입 → 이를 통해 **모델의 자연스러움 향상**과 **모델 파라미터 수 감소**라는 두 가지 효과

RLHF 학습 방식 3단계

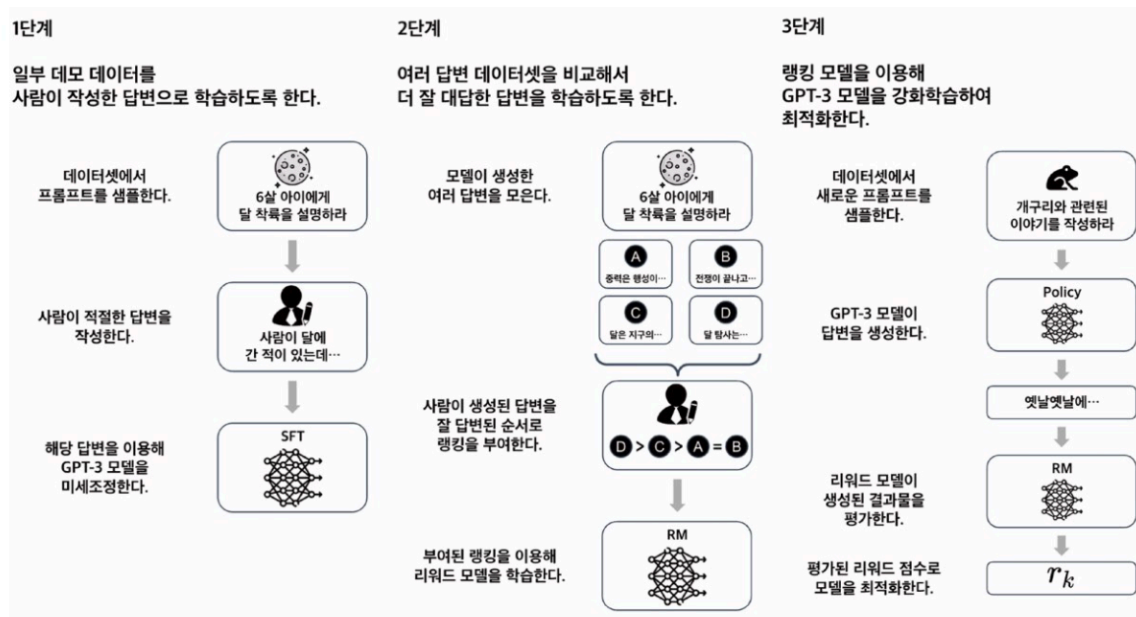


그림 7.12 RLHF GPT-3 학습 방법

- **1단계: 지도 학습 (SFT, Supervised Fine-Tuning)**
 - 사람이 작성한 정답 데이터를 활용해 모델이 정답을 **직접 모방 학습**
 - 이 단계에서는 GPT-3 모델에 사람의 답변을 **지도학습**으로 입력
- **2단계: 보상 모델(RM) 학습**
 - 동일한 입력에 대해 **여러 개의 답변을 생성**
 - 사람이 직접 이 답변들을 랭킹(Ranking)하여 우수한 순서로 정렬
 - 이 순서를 학습하여 보상 모델(Reward Model)**을 생성
- **3단계: 강화 학습 (PPO 등)**
 - 보상 모델을 사용하여 GPT-3 언어 모델을 **정책 최적화** 방식으로 학습
 - 즉, **우수한 답변일수록 높은 보상을** 받도록 강화학습

• GPT-3.5의 동작 특징

- 동일한 입력(Seed)에 대해서도 **여러 개의 답변**을 생성할 수 있음
- 보상 모델은 사람이 매긴 **랭킹 정보를 기반으로**, 생성된 문장의 자연스러움/품질을 학습
- 따라서 GPT-3.5는 GPT-3보다 **자연스러운 문장 생성, 더 안정적이고 일관된 출력**

- **실제 활용**
 - **사람의 피드백 기반 평가**로 학습되기 때문에, **실제 사용자의 의도와 일치하는 답변**을 생성할 가능성이 높음
 - 단, 입력 전체에 대해 사람이 랭킹을 매기는 것은 **비현실적**이므로 일부 데이터를 샘플링해 사용
- **주의할 점**
 - GPT-4와 비교해도 여전히 **높은 성능**을 보이지만, **환각(Hallucination)** 문제 존재
 - 실제 존재하지 않는 정보 생성, 그럴듯하지만 틀린 정보를 사실처럼 서술
 - 생성된 텍스트는 항상 **비판적으로 검토**가 필요

GPT-4 (MultiModal)

• 멀티모달(Multimodal) 모델

- GPT-4는 이전 GPT 시리즈와 달리, **텍스트뿐 아니라 이미지 데이터도 인식 가능한 멀티모달 모델**입니다.
- 즉, 단순 언어 모델을 넘어서 다양한 데이터 유형을 동시에 처리 가능

- **입력 용량 확장** : [GPT-3.5] 최대 입력 토큰 4096개 → [GPT-4] 32768개 (약 8배 증가)
 - (ex) GPT-3.5는 약 3,000개의 단어 입력 가능 → GPT-4는 약 25,000개의 단어 처리 가능
→ 긴 문서 이해, 분석, 요약 등에서 **훨씬 유리**

- 성능 평가 (MMLU 벤치마크 기준) → 전반적으로 **전 세대보다 정확도와 이해력 향상**

항목	GPT-3.5	GPT-4
MMLU (다중 작업 언어 이해)	약 70%	약 85%
한국어 MMLU	77%	영어보다 낮음 (언급)
미국 변호사 시험	213점, 상위 10% 미만	298점, 상위 10%
SAT 읽기/쓰기	670점	710점 (상위 7%)

- 모델의 비공개 요소
 - GPT-4의 **파라미터 수, 학습 데이터, 학습 방식**은 비공개
 - ChatGPT Plus** 사용자 또는 **Microsoft Bing**을 통해 GPT-4 기반 모델 사용 가능